JAIN DEEMED-TO-BE UNIVERSITY | ONLINE

E-Learning Material

Computer Architecture

(25VBT0C104)

www.onlinejain.com

Module: 01

Fundamentals of Computer Structure & Instruction Set

Module Information

Field	Details
Course Title	Computer Architecture
Course Code	25VBT0C104
Programme / Semester	BCA — Semester I
Course Type	Core Course
Credits	4
Module Number	Module 1
Module Title	Fundamentals of Computer Structure and Instruction Set
Learning Hours	24 Hours

Module Overview

Welcome to the world of Computer Architecture — one of the most fascinating and foundational subjects in the entire field of Computer Science! Every time you use a smartphone, open a laptop, or interact with any digital device, you are relying on the principles that this course will teach you. Understanding how a computer is built, how it processes instructions, and how it stores and retrieves data is the bedrock upon which all software, programming, networking, and AI applications are built.

This first module lays the essential foundation for the entire course. It begins at the very beginning — asking the fundamental question: what exactly is a computer, and what different types of computers exist? You will then be introduced to the functional units that together make a computer work — the processor, memory, input and output devices — and understand how they communicate with each other through bus structures.

A large and critically important part of this module is dedicated to number systems — the language that computers speak. Unlike humans, who naturally use the decimal (base-10) system, computers use binary (base-2), octal (base-8), and hexadecimal (base-16) number systems. You will learn not only how to represent numbers in these systems, but also how computers perform arithmetic operations on both positive and negative numbers — a topic that is fundamental to understanding how a processor works at its lowest level.

The module also covers instruction formats — the structured way in which a computer is told what to do — and memory organisation — how data is stored at specific locations and how the processor reads and writes that data. By the end of this module, you will have a clear mental model of how a computer functions, expressed in the precise technical language of computer architecture. This foundation will serve you through every subsequent module of this course and across your entire BCA programme.

Learning Objectives

By the end of this module, you will be able to:

- Identify the different types of computers and describe the purpose of each type. (BTL 1)
- Explain the role of each functional unit of a computer and describe how they work together. (BTL 2)
- Describe the basic operational concept of a computer — the fetch-decode-execute cycle. (BTL 2)
- Convert numbers between binary, octal, decimal, and hexadecimal number systems. (BTL 2)
- Represent signed integers using signed magnitude, 1s complement, and 2s complement notation. (BTL 2)

Learning Outcomes

Upon successful completion of this module, you will be able to:

- List at least five types of computers and explain one real-world application for each.
- Draw and label a diagram of the functional units of a computer and explain the role of each unit.
- Trace the execution of a simple instruction through the fetch-decode-execute cycle step by step.
- Perform number system conversions between binary, octal, decimal, and hexadecimal correctly.
- Represent positive and negative integers in 2s complement notation and perform 2s complement addition and subtraction.

Table of Topics

Unit	Topic / Sub-Unit	Key Concepts Covered
1.1	Introduction to Computers and Their Types	Definition, classification: supercomputer, mainframe, minicomputer, microcomputer, embedded systems
1.2	Functional Units of a Computer	CPU, ALU, CU, registers, memory unit, I/O units, interconnections
1.3	Basic Operational Concepts	Fetch-decode-execute cycle, instruction execution, stored-program concept
1.4	Bus Structures	Data bus, address bus, control bus, single-bus vs. multiple-bus organisation
1.5	Instruction Formats	Zero, one, two, three-address instruction formats, opcode, operand fields
1.6	Number Systems and Data Representation	Binary, octal, hexadecimal, conversions, signed magnitude, 1s complement, 2s complement
1.7	Arithmetic Operations on Signed and Unsigned Numbers	Binary addition, subtraction, overflow detection, 2s complement arithmetic
1.8	Memory Locations and Addressing	Memory address, word size, byte addressing, big-endian vs. little-endian
1.9	Memory Read and Write Operations	Memory access cycle, MAR, MDR, read/write control signals, timing

1.1 Introduction to Computers and Their Types

A computer is an electronic device that accepts data as input, processes it according to a set of instructions (called a program), stores intermediate and final results, and produces output. This simple definition contains four fundamental operations that every computer — from the most powerful supercomputer to the smallest microcontroller in your washing machine — performs: Input, Processing, Storage, and Output (IPSO). Understanding computers begins with understanding these four operations and how they are organised.

The word 'computer' originally referred to a human being who performed calculations by hand. In the 1940s, the term began to be applied to electronic machines capable of performing these calculations automatically and at extraordinary speed. Since then, computers have evolved through several generations — from room-sized vacuum-tube machines to nanometre-scale transistors — but the fundamental architectural principles introduced in the early days remain relevant today.

Key Concept

A computer is an electronic, programmable, general-purpose device that automatically executes stored

sequences of instructions (programs) to process data and produce useful output. The key word is

"programmable" — unlike a calculator with fixed operations, a computer can be reprogrammed to

perform any computable task simply by loading a different program.

Classification of Computers

Computers are classified in several ways — by size, processing power, purpose, and cost. The most common classification is by size and capability:

1. Supercomputers

Supercomputers are the most powerful computers in existence. They are designed for tasks that require enormous amounts of computational power — such as weather forecasting, nuclear simulation, molecular modelling, and astronomical calculations. They use thousands of processors working in parallel and can perform hundreds of quadrillions of calculations per second (measured in Petaflops).

- Example: PARAM Siddhi-AI (India) — ranked among the top 500 supercomputers globally. Developed by the Centre for Development of Advanced Computing (C-DAC) in Pune, it is used for AI research, drug discovery, and climate modelling.
- Speed: Hundreds of Petaflops (10^15 floating point operations per second).
- Cost: Hundreds of crores of rupees. Used exclusively by governments, research institutions, and large corporations.

2. Mainframe Computers

Mainframes are large, powerful computers designed to process vast amounts of data and support hundreds or thousands of simultaneous users. They are the backbone of large organisations such as banks, airlines, insurance companies, and government departments. Unlike supercomputers (which focus on a single enormous computation), mainframes excel at handling many smaller transactions simultaneously and with extraordinary reliability.

- Example: IBM Z-Series mainframes are used by the State Bank of India and major Indian banks to process millions of daily transactions with near-zero downtime.
- Key strength: Reliability, security, and the ability to run multiple operating systems simultaneously.

3. Minicomputers (Midrange Computers)

Minicomputers occupy the middle ground between mainframes and personal computers. They support multiple users and are used by medium-sized businesses, universities, and research laboratories. The term 'mini' reflected their smaller size compared to mainframes when the category was introduced in the 1960s — though by today's standards they would be considered quite large. In modern usage, the equivalent category is often called 'midrange servers'.

- Example: IBM AS/400 (now IBM iSeries) systems used by medium-sized Indian manufacturing companies and educational institutions.

4. Workstations

Workstations are high-performance single-user computers designed for technical and scientific applications. They have more processing power, memory, and graphics capability than ordinary personal computers, and are used by engineers, architects, animators, video editors, and researchers.

- Example: Workstations running CAD software (AutoCAD, CATIA) used by mechanical engineers at Tata Motors or Mahindra for vehicle design.

5. Personal Computers (Microcomputers)

Personal computers (PCs) are designed for individual use by a single person. They include desktop computers, laptop computers, tablets, and smartphones. The term

'microcomputer' reflects the fact that the central processor is a single microprocessor chip. Personal computers revolutionised computing by making it affordable and accessible to ordinary people.

- Desktop PCs: Used for home computing, office applications, gaming, programming.
- Laptop/Notebook: Portable versions of desktop PCs — widely used by students and professionals.
- Tablets and Smartphones: Handheld computers with touch interfaces — the most numerous computers on the planet.

6. Embedded Computers

Embedded computers are special-purpose computers built into other devices to control or monitor specific functions. Unlike general-purpose computers, embedded systems run a fixed program that is stored permanently in their memory. They are everywhere — in cars, household appliances, medical devices, industrial machinery, and consumer electronics.

- Examples in daily life: The microcontroller in a washing machine (controls wash cycles), the engine management unit in a car, the processor in a digital camera, the controller in a traffic light system.
- Scale: There are estimated to be over 30 billion embedded processors deployed worldwide — vastly outnumbering general-purpose computers.

Type	Speed / Power	Users	Cost (approx.)	Indian Example
Supercomputer	Petaflops	Researchers	100s of Crores	PARAM Siddhi-AI, Pune
Mainframe	Teraflops	1000s simultaneous	10s of Crores	SBI banking systems
Minicomputer	Gigaflops	100s simultaneous	Lakhs to Crores	University servers
Workstation	High GHz, multi-core	1 (power user)	1–5 Lakhs	Tata Motors CAD labs
Personal Computer	GHz (consumer)	1 individual	30K–1.5 Lakhs	Home/office PCs
Embedded System	MHz to GHz	Dedicated device	Rs. 50 – Rs. 5000	Washing machine MCU

1.2 Functional Units of a Computer

Regardless of the type or size of a computer — whether a smartphone or a supercomputer — every computer is built from the same fundamental building blocks, called functional units. These units each perform a distinct role, and together they enable the computer to carry out any computation. Understanding these units and how they interact is the most important conceptual step in learning computer architecture.

The five major functional units of a computer are: the Input Unit, the Output Unit, the Memory Unit, the Arithmetic and Logic Unit (ALU), and the Control Unit (CU). The ALU and the Control Unit together form the Central Processing Unit (CPU) — the 'brain' of the computer. Let us examine each unit in detail.

1. Input Unit

The Input Unit accepts data and instructions from the external world and converts them into a binary form that the computer can process. Input devices translate human-readable or real-world information (keystrokes, images, sound, temperature readings) into binary electrical signals.

- Examples of input devices: Keyboard, mouse, scanner, microphone, camera, touchscreen, barcode reader, temperature sensor (in embedded systems), network interface card.
- The input unit's role is purely receptive — it accepts data and passes it into the computer's memory for processing.

2. Memory Unit

The Memory Unit stores data and instructions — both the program currently being executed and the data it is working on. Computer memory is organised as a large collection of storage locations, each with a unique numerical address. Memory is divided into two broad categories:

- Primary Memory (Main Memory): Directly accessible by the CPU. Includes RAM (Random Access Memory — volatile, fast, temporary) and ROM (Read-Only Memory — non-volatile, stores firmware). Also includes processor registers and cache memory (ultra-fast, very small).
- Secondary Memory (Storage): Larger, slower, non-volatile storage. Includes hard disk drives (HDD), solid-state drives (SSD), USB flash drives, and optical discs (CD/DVD). Not directly accessed by the CPU — data must be loaded into primary memory first.

Key Concept

The fundamental distinction between primary and secondary memory:

PRIMARY MEMORY (RAM): Fast, expensive, volatile (loses data when power is off), directly accessible

by the CPU. Small in capacity (typically 4 GB – 64 GB in a modern PC).

SECONDARY MEMORY (HDD/SSD): Slow, inexpensive, non-volatile (retains data without power),

not directly accessible by CPU. Large in capacity (typically 256 GB – several TB).

The CPU can only work on data that is currently in primary memory (RAM).

3. Arithmetic and Logic Unit (ALU)

The ALU is the computational engine of the computer — the unit that performs all arithmetic operations (addition, subtraction, multiplication, division) and all logical operations (AND, OR, NOT, XOR, comparison). Everything that a computer 'calculates' happens in the ALU. It is implemented using electronic logic circuits (gates) built from transistors on the processor chip.

- Arithmetic Operations: Addition (+), Subtraction (-), Multiplication (*), Division (/), and their variations.
- Logical/Comparison Operations: AND, OR, NOT, XOR, NAND, NOR; comparisons such as equal-to, greater-than, less-than.
- The ALU takes operands (data values) from registers (fast temporary storage locations inside the CPU), performs the specified operation, and deposits the result back into a register or memory.

4. Control Unit (CU)

The Control Unit is the manager or director of the computer. It does not perform calculations itself — instead, it orchestrates all the other units by fetching instructions from memory, decoding them to determine what action is required, and issuing the appropriate control signals to the ALU, memory, and I/O units to carry out the instruction. The Control Unit follows a rigidly defined sequence called the Instruction Cycle (or Fetch-Decode-Execute cycle), which is described in detail in section 1.3.

- The CU generates timing and control signals that coordinate the activities of all other units.
- It reads instructions from memory and interprets them (decodes the opcode — the operation code — and operand addresses).
- It controls the flow of data between memory, the ALU, and the I/O units.

5. Output Unit

The Output Unit receives the results of processing from the computer and converts them into a human-readable or machine-usable form. Like the input unit, the output unit serves as a translator — this time from binary signals inside the computer to something meaningful in the external world.

- Examples of output devices: Monitor/display screen, printer, speaker, projector, motor controller (in embedded systems), network interface card (for transmitting data).

The CPU: ALU + Control Unit

The Central Processing Unit (CPU) is the combination of the ALU and the Control Unit, plus a set of registers — small, ultra-fast storage locations directly inside the processor. The CPU is the component colloquially called the 'brain' of the computer. It is responsible for executing program instructions. Key registers inside the CPU include:

Register Name	Function
Program Counter (PC)	Holds the memory address of the NEXT instruction to be fetched. Automatically increments after each fetch.
Instruction Register (IR)	Holds the instruction currently being decoded and executed.
Memory Address Register (MAR)	Holds the address in memory that the CPU wants to read from or write to.
Memory Data Register (MDR)	Holds the data being transferred to or from memory. Also called the Memory Buffer Register (MBR).
Accumulator (ACC)	Stores the result of ALU operations in many processor designs. The accumulator is the primary working register for computations.
Status / Flag Register	Holds condition codes (flags) set by ALU operations: Zero flag (result was zero), Carry flag (arithmetic carry), Overflow flag, Sign flag (result negative).

1.3 Basic Operational Concepts

To understand how a computer actually works — not just what its parts are, but how they cooperate to execute a program — you must understand two foundational ideas: the stored-program concept, and the instruction execution cycle.

The Stored-Program Concept

Before modern computers, calculating machines were built for a specific purpose and could not be reprogrammed. The revolutionary idea that transformed computing was proposed by mathematician John von Neumann in 1945: both the program (instructions) and the data it operates on should be stored together in the same memory, in the same format (binary numbers). This is called the stored-program concept, and it is the architectural principle underlying virtually every computer built since.

The implications are profound: to change what a computer does, you simply load a different program into memory — you do not need to rewire the hardware. This is what makes a computer 'general-purpose'.

Did You Know?

The design principle of storing both program instructions and data in the same binary memory is called the

'von Neumann architecture', after mathematician John von Neumann (1903–1957). However, historical research

has shown that British engineer J. Presper Eckert had the same idea independently. The first computer to

implement this concept was EDSAC (Electronic Delay Storage Automatic Calculator) at Cambridge University,

UK, in 1949. Every laptop, smartphone, and server you use today follows this 75-year-old principle.

The Fetch-Decode-Execute Cycle

The CPU executes programs by continuously repeating a three-step cycle: Fetch, Decode, and Execute. This cycle is also called the instruction cycle or machine cycle, and it happens billions of times per second in a modern processor (a 3 GHz processor executes up to 3 billion cycles per second).

Step 1: FETCH

The Control Unit reads (fetches) the next instruction from the memory location pointed to by the Program Counter (PC). The address in the PC is copied to the Memory Address Register (MAR). The memory system is accessed, and the instruction at that address is read into the Memory Data Register (MDR), then transferred to the Instruction Register (IR). The PC is then incremented to point to the next instruction.

Step 2: DECODE

The Control Unit examines the instruction in the Instruction Register (IR) and determines what operation is to be performed. The instruction is divided into fields: the opcode (which specifies the operation — e.g., ADD, LOAD, STORE, JUMP) and the operand address(es) (which specify where the data is). Decoding involves routing the appropriate control signals to the relevant hardware units.

Step 3: EXECUTE

The operation specified by the decoded instruction is carried out. If it is an arithmetic operation (e.g., ADD), the operands are fetched from their specified locations into the ALU, the operation is performed, and the result is stored. If it is a memory operation (LOAD or STORE), data is transferred between the CPU and memory. If it is a branch (JUMP), the PC is updated to the new address.

Worked Example: Tracing the Fetch-Decode-Execute Cycle

Consider a simple instruction: ADD R1, R2 (Add the value in Register R2 to Register R1)

FETCH:

1. PC = 1000 (holds address of next instruction)
2. MAR ← 1000 (copy address to MAR)
3. Memory reads instruction at address 1000 into MDR
4. IR ← MDR (instruction is now in IR)
5. PC ← 1001 (increment PC to next instruction)

DECODE:

6. CU reads IR: opcode = ADD, source = R2, destination = R1
7. CU signals ALU to prepare for addition
8. CU signals register file to route R1 and R2 to ALU inputs

EXECUTE:

9. ALU receives values from R1 and R2
10. ALU computes: result = R1 + R2
11. Result is stored back into R1
12. Status flags are updated (zero flag, carry flag, etc.)

→ Cycle complete. Return to Step 1 (FETCH next instruction at PC = 1001).

1.4 Bus Structures

Inside a computer, the various functional units — CPU, memory, and I/O devices — must constantly exchange data, addresses, and control signals. The communication pathway that carries these signals between components is called a bus. A bus is a shared communication channel: a set of parallel electrical conductors (wires or PCB traces) that multiple devices are connected to simultaneously. Understanding bus structures is essential to understanding how the computer's units communicate and why performance can be limited by the communication channel itself.

Types of Buses

There are three functionally distinct buses in a computer system:

1. Data Bus

The data bus carries the actual data being transferred between the CPU, memory, and I/O devices. It is bidirectional — data can flow in either direction (from memory to CPU when reading, or from CPU to memory when writing). The width of the data bus (measured in bits) is one of the most important characteristics of a processor: a 32-bit data bus transfers 32 bits simultaneously; a 64-bit bus transfers 64 bits. Wider buses mean more data can be transferred per clock cycle, improving performance.

- A 64-bit data bus (as found in all modern 64-bit processors) can transfer 8 bytes per bus cycle.

2. Address Bus

The address bus carries the memory address — the location in memory that the CPU wants to read from or write to. It is unidirectional — addresses flow only from the CPU to memory (or I/O devices); memory does not send addresses back to the CPU. The

width of the address bus determines the maximum amount of memory the processor can access.

- Formula: Maximum addressable memory = 2^n bytes, where n = number of address bus lines.
- Example: A 32-bit address bus can address 2^32 = 4,294,967,296 bytes = 4 GB of memory. A 64-bit address bus (in practice, modern systems use 48 bits) can address 2^48 = 256 TB.

3. Control Bus

The control bus carries control signals — commands and status information that coordinate the activities of all components. Unlike the data and address buses, the control bus is not a single set of lines carrying a single value; it is a collection of individual control lines, each with a specific purpose.

- Memory Read (MR): Signals memory to place data at the specified address onto the data bus.
- Memory Write (MW): Signals memory to accept data from the data bus and store it at the specified address.
- I/O Read (IOR): Read from an I/O device.
- I/O Write (IOW): Write to an I/O device.
- Interrupt Request (IRQ): An I/O device signals the CPU that it needs attention.
- Clock: Synchronisation signal that coordinates timing across all components.

Worked Example: Address Bus Width and Addressable Memory

Address Bus Width	Maximum Addressable Memory	Example System
16 bits	2^16 = 65,536 bytes = 64 KB	Intel 8086 (1978)
20 bits	2^20 = 1,048,576 bytes = 1 MB	Intel 8088 (IBM PC, 1981)
24 bits	2^24 = 16 MB	Motorola 68000 (1979)
32 bits	2^32 = 4 GB	Intel 80386 (1985)
64 bits	2^64 = 16 Exabytes (theoretical)	Modern 64-bit processors

Note: Modern 64-bit processors typically use 48 bits of address = 256 TB physical.

Single-Bus vs. Multiple-Bus Organisation

In a single-bus organisation, all components (CPU, memory, and all I/O devices) are connected to a single shared bus. While simple and inexpensive, this creates a bottleneck: only one device can use the bus at a time. If the CPU is communicating with memory, all I/O devices must wait.

In a multiple-bus organisation, different buses serve different communication needs. A common arrangement uses a high-speed local bus (front-side bus or backside bus) directly between the CPU and main memory, and a separate slower I/O bus for peripheral devices. Modern computers typically use multiple specialised buses (PCI Express for graphics, SATA for storage, USB for peripherals) to avoid bottlenecks and match the speed of each bus to the speed requirements of the connected devices.

1.5 Instruction Formats

An instruction is a command given to the CPU, expressed as a binary pattern that the processor can decode and execute. Every instruction consists of two parts: the opcode (operation code), which specifies what operation to perform, and zero or more operands, which specify the data to operate on or the addresses where data can be found. The way these fields are arranged and how many operands an instruction contains defines the instruction format.

Different computer architectures use different instruction formats. The choice of format involves trade-offs between code density (how many instructions fit in a given memory space), simplicity of hardware, and flexibility of expression. The number of operand addresses in an instruction format gives rise to the classification of instruction types.

Key Concept

Every machine instruction = OPCODE + OPERAND(s)

OPCODE: A binary code that identifies the operation (ADD, SUB, LOAD, STORE, JUMP, etc.)

OPERAND: The data to be operated on, or the address where data is located.

The number of operand fields in the instruction format defines the instruction type.

Zero-Address Instructions

In a zero-address (0-address) instruction, there are no explicit operand addresses in the instruction. Instead, the operands are implicitly on a stack — a last-in, first-out

(LIFO) data structure. The CPU always operates on the top one or two elements of the stack. This design is used in stack-based architectures (e.g., the Java Virtual Machine).

- Format: | OPCODE |
- Example: ADD — pops the top two values from the stack, adds them, and pushes the result back.
- Advantage: Very compact instructions (only the opcode is needed).
  Disadvantage: Frequent stack push/pop operations; less flexible.

One-Address Instructions

In a one-address (1-address) instruction, there is one explicit operand address. The other operand is implicitly the accumulator register — a special register in the CPU that acts as the primary storage for computations. The result is also stored back in the accumulator. This was common in early computers and is still found in some microcontrollers.

- Format: | OPCODE | ADDRESS |
- Example: ADD X — adds the value at memory address X to the accumulator. Result stored in accumulator.
- The full operation: ACC ← ACC + Memory[X]

Two-Address Instructions

In a two-address (2-address) instruction, there are two explicit operand addresses. The result overwrites one of the source operands (usually the first/destination operand). This is the most common instruction format in modern CISC (Complex Instruction Set Computer) architectures, including the Intel x86 family.

- Format: | OPCODE | DESTINATION | SOURCE |

- Example: ADD R1, R2 — adds the value in register R2 to register R1, result stored in R1.
- The full operation: R1 ← R1 + R2 (R1 is both a source and the destination).

Three-Address Instructions

In a three-address (3-address) instruction, there are three explicit operand addresses — two source operands and a separate destination address. This allows the source operands to be preserved after the operation, which is more flexible and avoids the need to save and restore intermediate values. Common in RISC (Reduced Instruction Set Computer) architectures like ARM and MIPS.

- Format: | OPCODE | DESTINATION | SOURCE1 | SOURCE2 |
- Example: ADD R3, R1, R2 — adds R1 and R2, stores result in R3. R1 and R2 are unchanged.
- The full operation: R3 ← R1 + R2

Type	Format	Example	Operation
0-Address	*OPCODE*	*ADD*	TOS ← TOS + NOS (stack)
1-Address	*OPCODE ADDR*	*ADD X*	ACC ← ACC + Mem[X]
2-Address	*OPCODE DST SRC*	*ADD R1, R2*	R1 ← R1 + R2
3-Address	*OPCODE DST S1 S2*	*ADD R3, R1, R2*	R3 ← R1 + R2

✓ Let's Check 1	
Q.No	**Question**
1	MCQ: The Control Unit of a CPU is responsible for: a) Performing arithmetic calculations. b) Storing the result of computation permanently. c) Fetching, decoding instructions and issuing control signals. d) Converting binary data to human-readable output.
2	MCQ: A 24-bit address bus can directly address a maximum of: a) 24 MB of memory b) 16 MB of memory c) 64 KB of memory d) 4 GB of memory
3	Fill in the Blank: The register that holds the address of the NEXT instruction to be executed is called the _____ (PC).
4	Fill in the Blank: In a two-address instruction "ADD R1, R2", the operation performed is R1 ← R1 __ R2.
5	True/False: The data bus in a computer system is unidirectional — it carries data only from memory to the CPU.
6	True/False: The stored-program concept means that both the program instructions and the data are stored together in the same memory in binary format.
Answers	
1	c) Fetching, decoding instructions and issuing control signals. The ALU performs arithmetic; the CU directs and coordinates.

2	b) 16 MB. Maximum addressable memory = 2^24 = 16,777,216 bytes = 16 MB.
3	Program Counter
4	+ (addition). R1 ← R1 + R2. R1 is both a source and destination.
5	False — The data bus is BIDIRECTIONAL. Data flows from memory to CPU (read) and from CPU to memory (write).
6	True — This is the foundational von Neumann stored-program concept.

1.6 Number Systems and Data Representation

One of the most important skills you will develop in this course is the ability to work fluently with the number systems that computers use. Humans naturally use the decimal (base-10) system because we have ten fingers. Computers, however, are built from electronic switches that have only two states — ON and OFF — which naturally correspond to the digits 1 and 0. This is why computers work in binary (base-2). Two other number systems — octal (base-8) and hexadecimal (base-16) — are widely used as convenient shorthand for binary, because they are easy for humans to read and write while being directly convertible to binary.

The Four Number Systems

System	Base (Radix)	Digits Used	Why It Matters
Decimal	10	0, 1, 2, 3, 4, 5, 6, 7, 8, 9	Human natural system; used in everyday life.
Binary	2	0, 1	Native language of computers. All data is stored and processed as binary.
Octal	8	0, 1, 2, 3, 4, 5, 6, 7	Compact notation for binary. Used in Unix/Linux file permissions.
Hexadecimal	16	0–9, A, B, C, D, E, F	Widely used in programming, memory addresses, colour codes. 1 hex digit = 4 bits.

Place Value (Positional Notation)

In any number system with base r, the value of a number is determined by the position (place) of each digit. Each position represents a power of the base, starting from r^0 (= 1) at the rightmost position and increasing leftward. This is called positional notation or the weighted positional system.

Worked Example: Understanding Place Value in Different Bases

DECIMAL: 2025

= 2×10^3 + 0×10^2 + 2×10^1 + 5×10^0

= 2×1000 + 0×100 + 2×10 + 5×1

= 2000 + 0 + 20 + 5 = 2025

BINARY: 1101 (reading rightmost digit as 2^0)

= 1×2^3 + 1×2^2 + 0×2^1 + 1×2^0

= 1×8 + 1×4 + 0×2 + 1×1

= 8 + 4 + 0 + 1 = 13 (in decimal)

HEXADECIMAL: 2F

= 2×16^1 + F×16^0 (F = 15)

= 2×16 + 15×1

= 32 + 15 = 47 (in decimal)

Number System Conversions

Being able to convert between number systems is a fundamental skill in computer science. The table below summarises the decimal equivalents of the first 16 values across all four systems:

Decimal	Binary	Octal	Hexadecimal
0	*0000*	0	0
1	*0001*	1	1
2	*0010*	2	2
3	*0011*	3	3
4	*0100*	4	4
5	*0101*	5	5
6	*0110*	6	6
7	*0111*	7	7
8	*1000*	10	8
9	*1001*	11	9
10	*1010*	12	**A**
11	*1011*	13	**B**
12	*1100*	14	**C**
13	*1101*	15	**D**
14	*1110*	16	**E**
15	*1111*	17	**F**

Decimal to Binary Conversion — Repeated Division by 2

To convert a decimal integer to binary: repeatedly divide the number by 2 and record the remainder. Read the remainders from bottom to top.

Worked Example: Convert Decimal 45 to Binary

45 ÷ 2 = 22 remainder 1 ← Least Significant Bit (LSB)

22 ÷ 2 = 11 remainder 0

11 ÷ 2 = 5 remainder 1

5 ÷ 2 = 2 remainder 1

2 ÷ 2 = 1 remainder 0

1 ÷ 2 = 0 remainder 1 ← Most Significant Bit (MSB)

Read remainders from bottom to top: 45 (decimal) = 101101 (binary)

Verification: 1×32 + 0×16 + 1×8 + 1×4 + 0×2 + 1×1 = 32+8+4+1 = 45 ✓

Binary to Hexadecimal Conversion — Group by 4 Bits

Converting binary to hexadecimal is easy: group the binary digits into sets of 4 (from the right), and replace each group of 4 bits with its hexadecimal digit. This is why hexadecimal is so useful in computing — it is a compact notation for binary.

Worked Example: Convert Binary 10110100 to Hexadecimal

Step 1: Group into sets of 4 bits from the right:

1011 | 0100

Step 2: Convert each 4-bit group to hex:

1011 = 8+2+1 = 11 = B (hex)

0100 = 4 = 4 = 4 (hex)

Result: 10110100 (binary) = B4 (hexadecimal)

Verification: B4 (hex) = 11×16 + 4 = 176 + 4 = 180 (decimal)

Binary check: 10110100 = 128+32+16+4 = 180 ✓

Representing Negative Numbers (Signed Number Representations)

In everyday mathematics, we use a minus sign (−) to indicate negative numbers. But inside a computer, all data is stored as binary bits — there is no minus sign. Instead, engineers have developed three methods for representing negative integers in binary. Each method uses the most significant bit (MSB) as a sign indicator (0 = positive, 1 = negative) in a fixed-width (n-bit) representation.

Method 1: Signed Magnitude

In signed magnitude representation, the MSB is the sign bit (0 for positive, 1 for negative), and the remaining bits represent the magnitude (absolute value) of the number — exactly as humans write negative numbers.

Worked Example: Signed Magnitude — 8-bit Examples

+45 → 0 1 0 1 1 0 1 → 0 0 1 0 1 1 0 1 (MSB=0, positive)

Sign|← Magnitude →|

-45 → 1 0 1 0 1 1 0 1 (MSB=1 means negative, magnitude = 0101101 = 45)

Range for 8 bits: -127 to +127

(Two representations of zero: 00000000 = +0 and 10000000 = -0)

Problem: Arithmetic with signed magnitude is complicated. Addition requires checking signs separately. Also, two representations of zero is wasteful.

Method 2: One's Complement (1's Complement)

In 1's complement, a positive number is represented normally. Its negative is obtained by inverting (flipping) every bit — 0 becomes 1, and 1 becomes 0. This makes some arithmetic operations simpler, but there is still the problem of two representations of zero.

Worked Example: 1's Complement — 8-bit Examples

+45 = 00101101 (same as signed magnitude)

-45: Invert all bits of +45:

0 0 1 0 1 1 0 1

→ 1 1 0 1 0 0 1 0 = -45 in 1s complement

Range for 8 bits: -127 to +127 (still two zeros: 00000000 and 11111111)

Method 3: Two's Complement (2's Complement) — The Standard Used by All Modern Computers

Two's complement is the representation used by virtually all modern computers because it has crucial advantages: there is only one representation of zero, arithmetic operations (addition, subtraction) work correctly without any special handling of signs, and hardware implementation is simpler and cheaper. To find the 2's complement of a number, invert all bits (find the 1's complement) and then add 1.

Worked Example: 2's Complement — Finding -45 in 8-bit representation

Step 1: Write +45 in 8-bit binary:

+45 = 0 0 1 0 1 1 0 1

Step 2: Find the 1's complement (invert all bits):

0 0 1 0 1 1 0 1

→ 1 1 0 1 0 0 1 0

Step 3: Add 1 to the 1's complement:

```
  1 1 0 1 0 0 1 0
+             1
_____________________
  1 1 0 1 0 0 1 1
```

Therefore: -45 in 2's complement (8-bit) = 11010011

Range for n-bit 2s complement: -2^(n-1) to +(2^(n-1) - 1)

For 8-bit: -128 to +127

For 16-bit: -32,768 to +32,767

For 32-bit: -2,147,483,648 to +2,147,483,647

🔍 Analysing Misconceptions: Two's Complement

✗ MYTH: To find the 2's complement of a number, you always flip all bits and add 1, even for zero or already-positive numbers.

✓ REALITY: The 2's complement representation is used to represent NEGATIVE numbers. For positive numbers, the representation is the same as regular binary. The flip-and-add-1 operation converts a positive binary number to its negative 2's complement equivalent, or converts a 2's complement negative back to positive. Also, the 2's complement of 0 is 0 itself: 00000000 → invert → 11111111 → +1 → 100000000 → drop the carry → 00000000. This confirms there is only one representation of zero.

1.7 Arithmetic Operations on Signed and Unsigned Numbers

Now that you understand how numbers are represented in binary and how negative numbers are encoded using 2's complement, you are ready to learn how the ALU actually performs arithmetic. The elegance of 2's complement representation is most clearly seen in arithmetic: addition and subtraction of both positive and negative numbers can be performed using the same hardware (an adder circuit), with no need to check signs separately.

Binary Addition — Basic Rules

Binary addition follows the same rules as decimal addition, but with only two digits (0 and 1). When the sum of two digits exceeds 1, a carry is generated to the next higher bit position:

- 0 + 0 = 0 (no carry)
- 0 + 1 = 1 (no carry)
- 1 + 0 = 1 (no carry)
- 1 + 1 = 10 (result = 0, carry = 1)
- 1 + 1 + 1 = 11 (result = 1, carry = 1) — occurs when there is a carry from the previous position

Worked Example: Binary Addition: 45 + 28 = 73

45 (decimal) = 0 0 1 0 1 1 0 1 (binary)

+ 28 (decimal) = 0 0 0 1 1 1 0 0 (binary)

Carry: 0 0 0 1 1 0 0 0

Sum: 0 1 0 0 1 0 0 1 (binary)

Column-by-column (right to left):

bit 0: 1+0 = 1, carry = 0

bit 1: 0+0 = 0, carry = 0

bit 2: 1+1 = 0, carry = 1

bit 3: 1+1+1 = 1, carry = 1

bit 4: 0+1+1 = 0, carry = 1

bit 5: 1+0+1 = 0, carry = 1

bit 6: 0+0+1 = 1, carry = 0

bit 7: 0+0 = 0, carry = 0

Result = 01001001 (binary) = 64+8+1 = 73 (decimal) ✓

Subtraction Using 2's Complement

One of the most important advantages of 2's complement is that subtraction can be performed using addition: to subtract B from A, simply add A and the 2's complement of B. The hardware only needs an adder — no subtractor circuit is needed. This simplifies processor design enormously.

Rule: A - B = A + (2's complement of B)

Worked Example: Subtraction Using 2's Complement: 45 - 28 = 17

Step 1: Express both numbers in 8-bit binary:

+45 = 00101101

+28 = 00011100

Step 2: Find 2's complement of 28 (negate it):

00011100 → invert → 11100011

11100011 → add 1 → 11100100 (this is -28 in 2s complement)

Step 3: Add 45 + (-28) using binary addition:

00101101 (+45)

+ 11100100 (-28 in 2s complement)

———————

100010001

Step 4: Discard the carry out (the 9th bit — leftmost 1):

Result = 00010001 (binary)

= 16 + 1 = 17 (decimal) ✓

Overflow Detection

Overflow occurs when the result of an arithmetic operation is too large to be represented in the fixed number of bits available. In 8-bit 2's complement arithmetic,

the valid range is -128 to +127. If the result falls outside this range, overflow has occurred, and the stored result is incorrect. The processor detects overflow by examining the carry bits:

- Overflow Rule: Overflow occurs when the carry INTO the sign bit (MSB) DIFFERS from the carry OUT OF the sign bit.
- Equivalently: Overflow occurs when you add two positive numbers and get a negative result, or add two negative numbers and get a positive result.

Worked Example: Overflow Detection: 100 + 50 = 150 (overflow!)

100 = 01100100 (8-bit)

\+ 50 = 00110010 (8-bit)

———————————

Carry: 00100000

Sum: 10010110 (binary)

MSB of result = 1 → interpreted as NEGATIVE in 2s complement = -106

But 100 + 50 should = 150, which exceeds 8-bit max of +127.

Overflow detection: Carry into MSB = 1, Carry out of MSB = 0 → DIFFERENT → OVERFLOW ✓

The Overflow (V) flag in the processor status register is set to 1.

1.8 Memory Locations and Addressing

Memory is the workplace of the computer — the space where programs are loaded and data is manipulated. Understanding how memory is organised and how specific locations within memory are identified and accessed is fundamental to understanding all aspects of computer architecture, from instruction execution to cache design to virtual memory.

Memory Structure

Computer memory can be visualised as a long sequence of storage cells, each capable of holding a fixed number of bits. Each cell has a unique numerical identifier called its address. The CPU communicates with memory by specifying an address (via the address bus) and then either reading data from that address or writing data to it.

Memory is almost universally organised in units of 8 bits (1 byte). Each byte has its own unique address. This is called byte-addressable memory. Most modern computers can access memory in units larger than one byte — typically 2 bytes (word), 4 bytes (doubleword), or 8 bytes (quadword) at a time, but the address always refers to the first (lowest-addressed) byte of the unit.

Key Concept

Key Memory Terminology:

BIT: The smallest unit of information — a single binary digit (0 or 1).

BYTE: 8 bits — the basic addressable unit of memory.

WORD: A group of bytes treated as a single unit by the processor.

Word size varies: 8-bit, 16-bit, 32-bit, or 64-bit processors have 1, 2, 4, or 8 byte words.

MEMORY ADDRESS: A unique binary number that identifies a specific byte in memory.

ADDRESS SPACE: The total range of addresses a processor can generate.

Byte Ordering: Big-Endian vs. Little-Endian

When a multi-byte value (such as a 32-bit integer) is stored in memory, the bytes must be arranged in some order across consecutive memory addresses. There are two conventions, and understanding them is essential for systems programming and network communication:

- Big-Endian: The most significant byte (MSB — the 'big end') is stored at the lowest address. The bytes are stored in the same order in which we write them — leftmost (most significant) first.
- Little-Endian: The least significant byte (LSB — the 'little end') is stored at the lowest address. The bytes are stored in reverse order — rightmost (least significant) byte first.

Worked Example: Big-Endian vs. Little-Endian Storage

Storing the 32-bit hexadecimal value 0x12345678 starting at address 1000:

BYTE BREAKDOWN:

Byte 0 (MSB): 0x12

Byte 1: 0x34

Byte 2: 0x56

Byte 3 (LSB): 0x78

BIG-ENDIAN (e.g., Motorola 68000, SPARC, IBM POWER):

Address 1000: 0x12 ← MSB first

Address 1001: 0x34

Address 1002: 0x56

Address 1003: 0x78 ← LSB last

LITTLE-ENDIAN (e.g., Intel x86, x64, ARM in LE mode):

Address 1000: 0x78 ← LSB first

Address 1001: 0x56

Address 1002: 0x34

Address 1003: 0x12 ← MSB last

Note: Intel/AMD processors (used in most PCs and laptops) are LITTLE-ENDIAN.

Network protocols (TCP/IP) use BIG-ENDIAN (called "network byte order").

Did You Know?

The terms "big-endian" and "little-endian" were coined by computer scientist Danny Cohen in a 1980 paper,

borrowing from Jonathan Swift's classic novel Gulliver's Travels, in which the citizens of Lilliput and

Blefuscu go to war over the politically divisive question of which end of a boiled egg should be opened

first. Cohen used the metaphor to highlight that the choice between big- and little-endian is ultimately

arbitrary — both are valid, but mixing them causes serious compatibility problems.

1.9 Memory Read and Write Operations

Understanding the memory read and write operations means understanding exactly how the CPU communicates with memory at the hardware level — what signals are sent, in what order, and what happens at each step. These operations are fundamental to the fetch-decode-execute cycle and to understanding how data is loaded into the CPU for processing and stored back to memory after processing.

The Memory Access Cycle

Every time the CPU reads from or writes to memory, it goes through a sequence of steps called the memory access cycle. The key components involved in this cycle are the Memory Address Register (MAR), the Memory Data Register (MDR, also called the Memory Buffer Register MBR), and the control signals on the control bus.

Memory READ Operation

A memory read operation transfers data from a specified memory location into the CPU (specifically into the MDR). The steps are:

- Step 1: The CPU places the address of the memory location it wants to read into the MAR.
- Step 2: The MAR places the address onto the address bus.
- Step 3: The Control Unit asserts the Memory Read (MR) control signal on the control bus.
- Step 4: The memory module receives the address, decodes it internally, and places the data stored at that address onto the data bus.
- Step 5: The data on the data bus is read into the MDR.
- Step 6: The data in the MDR is now available for the CPU to use (it may be copied to a register for further processing).

Memory WRITE Operation

A memory write operation transfers data from the CPU to a specified memory location. The steps are:

- Step 1: The CPU places the destination address into the MAR.
- Step 2: The CPU places the data to be written into the MDR.
- Step 3: The MAR places the address onto the address bus; the MDR places the data onto the data bus.
- Step 4: The Control Unit asserts the Memory Write (MW) control signal on the control bus.
- Step 5: The memory module receives the address and data, decodes the address, and stores the data at the specified location.

Worked Example: Memory Read/Write — Step-by-Step Signal Flow

READ: CPU wants to read the value stored at memory address 2050.

1. CPU: MAR ← 2050
2. Bus: Address Bus ← 2050
3. Bus: Control Bus ← MR (Memory Read signal asserted)
4. RAM: Decodes address 2050, places Memory[2050] on Data Bus
5. CPU: MDR ← Data Bus value
6. CPU: The data is now in MDR, ready for use.

WRITE: CPU wants to write the value 0x4F to memory address 3100.

1. CPU: MAR ← 3100
2. CPU: MDR ← 0x4F (data to be written)
3. Bus: Address Bus ← 3100
4. Bus: Data Bus ← 0x4F
5. Bus: Control Bus ← MW (Memory Write signal asserted)
6. RAM: Decodes address 3100, stores 0x4F at Memory[3100].

✓ Let's Check 2

Q.No	Question
1	MCQ: Which of the following correctly represents +45 in 8-bit 2's complement? a) 10101101 b) 00101101

	c) 11010011 d) 01010011
2	MCQ: In Little-Endian byte ordering, when storing the 32-bit value 0x12345678 at address 1000, which byte is stored at address 1000? a) 0x12 (Most Significant Byte) b) 0x34 c) 0x56 d) 0x78 (Least Significant Byte)
3	Fill in the Blank: To find the 2's complement of a binary number, first invert all bits (find the 1's complement), then _____.
4	Fill in the Blank: During a memory READ operation, the CPU places the memory address to be read into the _____ register.
5	True/False: Overflow occurs in 2's complement arithmetic when both operands are positive but the result has a 1 in the MSB (sign bit).
6	True/False: The Memory Data Register (MDR) holds the address of the memory location being accessed.
Answers	
1	b) 00101101. Positive numbers have MSB=0; 45 = 32+8+4+1 = 00101101. Option (c) 11010011 is the 2's complement of +45, which represents -45.
2	d) 0x78 (Least Significant Byte). In Little-Endian, the LEAST significant byte is stored at the LOWEST address.
3	Add 1. Rule: 2's complement = bitwise NOT (invert all bits) + 1.
4	MAR (Memory Address Register). The address goes into the MAR, which then places it on the address bus.

5	True — This is the definition of overflow in 2's complement: adding two positives yields a negative, or adding two negatives yields a positive.
6	False — The MDR (Memory Data Register) holds the DATA being transferred. The MAR (Memory Address Register) holds the ADDRESS.

Case Study

Number Representation in Practice: The Ariane 5 Rocket Failure (1996)

Background:

On 4 June 1996, the Ariane 5 rocket — developed by the European Space Agency (ESA) at a cost of approximately USD 7 billion over ten years — exploded just 37 seconds after launch. The loss included four scientific satellites worth USD 370 million. The cause was not a mechanical failure or fuel problem. It was a software bug rooted directly in the concept of number representation studied in this module: arithmetic overflow.

What Happened:

The Ariane 5's Inertial Reference System (IRS) — the on-board computer that tracks the rocket's speed and orientation — contained software originally written for the smaller Ariane 4 rocket. This software calculated a value called "horizontal bias" — a 64-bit floating-point number representing the rocket's horizontal velocity relative to Earth's rotation. It then attempted to convert this 64-bit floating-point value into a 16-bit signed integer.

The Problem — Overflow:

The Ariane 5 was a significantly more powerful rocket than the Ariane 4. Its higher speed meant that the horizontal bias value was much larger than Ariane 4's. When the software attempted to convert the large 64-bit number into a 16-bit signed integer, the value was too large to fit — it exceeded the maximum value representable in 16 bits (32,767 in 2's complement). An overflow occurred. The software had no exception handler for this specific conversion — it had been

reviewed for the Ariane 4 and found safe for that rocket's speed range, but the analysis was not repeated for Ariane 5.

The Consequence:

The overflow caused the IRS to crash and output an error code to the flight computer. The flight computer, receiving what it interpreted as flight data (not recognising it as an error code), made a violent steering correction. The rocket veered off course and was destroyed by the range safety officer to prevent it crashing into populated areas.

Connection to This Module:

1. Number Representation: The accident was directly caused by the limits of fixed-width signed integer representation — specifically the limited range of 16-bit 2's complement (-32,768 to +32,767). The horizontal bias value exceeded this range, causing overflow.

2. Overflow Detection: Modern processors set an overflow (V) flag when arithmetic overflow occurs. Software must check this flag and handle the exception. The Ariane 5 software failed to do this for one critical conversion.

3. Validation and Reuse: Software validated for one system cannot be assumed safe in another system with different operating parameters. The Ariane 4 IRS software was numerically safe for Ariane 4's speed range; Ariane 5's higher speed moved the values outside the safe range.

Learning Points:

- The consequences of arithmetic overflow can be catastrophic, not merely computational.

• Every fixed-width integer representation has a finite range; exceeding that range causes overflow.

• Software intended for safety-critical systems must validate all inputs against the representable range.

• 2's complement arithmetic is used in all real processors — understanding it is not academic; it is essential.

Source: Report of the Inquiry Board, ESA/ESOC, July 1996. Publicly available: https://www.ima.umn.edu/~arnold/disasters/ariane5rep.html

Summary

This module has laid the complete foundation for your study of Computer Architecture. You began with the big picture — understanding what computers are, how they are classified, and what different types of computers exist for different purposes. You then drilled down into the internal structure, learning the roles of each functional unit (Input, Output, Memory, ALU, Control Unit, and CPU) and how they cooperate to execute programs.

The fetch-decode-execute cycle gave you a precise mental model of what the CPU does at its lowest level — repeatedly fetching instructions from memory, decoding them, and executing them, billions of times per second. The bus structures module explained how these units communicate — the data bus carries data, the address bus carries memory addresses, and the control bus carries the timing and command signals that coordinate everything.

You then entered the world of number systems — the language of computers — learning to convert between decimal, binary, octal, and hexadecimal, and developing proficiency with the arithmetic these systems entail. The representation of negative numbers, culminating in the universally-used 2's complement notation, is one of the most practically important topics in the entire course. The case study of the Ariane 5 disaster illustrated with dramatic clarity why understanding overflow is not an abstract exercise but a matter of real-world consequence.

Key highlights from the module:

- Computers are classified as supercomputers, mainframes, minicomputers, workstations, personal computers, and embedded systems — each with different power, cost, and application domains.

- The five functional units are: Input, Output, Memory, ALU, and Control Unit. The CPU = ALU + CU + Registers.
- The fetch-decode-execute cycle: Fetch instruction from memory → Decode its opcode → Execute the operation. Key registers: PC, IR, MAR, MDR, ACC, Status Register.
- Three buses: Data bus (bidirectional, carries data), Address bus (unidirectional, carries addresses), Control bus (carries timing and command signals). Address bus width determines maximum addressable memory (2^n bytes).
- Instruction formats: 0-address (stack), 1-address (accumulator), 2-address (CISC), 3-address (RISC). Format: OPCODE + OPERAND(s).
- Number systems: Decimal (base 10), Binary (base 2), Octal (base 8), Hexadecimal (base 16). Conversions: Repeated division for decimal→binary; group by 4 bits for binary→hexadecimal.
- Signed number representations: Signed magnitude, 1's complement, and 2's complement. Modern computers use 2's complement. Rule: negate = invert all bits + 1.
- 2's complement arithmetic: Subtraction = addition of the negated value. Overflow detection: carry-in to MSB ≠ carry-out from MSB.
- Memory addressing: byte-addressable; big-endian (MSB at lowest address) vs. little-endian (LSB at lowest address). Intel/AMD = little-endian.
- Memory read: CPU → MAR → address bus → MR signal → data bus → MDR. Memory write: CPU → MAR + MDR → address+data buses → MW signal → memory stores data.

Glossary

Term	Definition
ALU (Arithmetic Logic Unit)	The functional unit of the CPU that performs all arithmetic operations (add, subtract) and logical operations (AND, OR, NOT). It is the computing engine of the processor.
Bus	A shared communication pathway — a set of parallel electrical conductors — that connects the CPU, memory, and I/O devices and transfers data, addresses, and control signals between them.
2's Complement	The standard method for representing signed integers in modern computers. To negate a number: invert all bits and add 1. Advantage: only one representation of zero; arithmetic requires no special sign handling.
Fetch-Decode-Execute Cycle	The fundamental operational cycle of a CPU: (1) Fetch the next instruction from memory into the IR, (2) Decode the opcode to determine the operation, (3) Execute the operation. Repeats continuously while the processor is running.
Program Counter (PC)	A CPU register that holds the memory address of the NEXT instruction to be fetched. After each fetch, the PC is automatically incremented to point to the following instruction.
Overflow	An error condition that occurs when the result of an arithmetic operation exceeds the representable range of the fixed-width integer format. Detected when carry-in to the sign bit differs from carry-out.

Opcode	The operation code portion of a machine instruction. It is a binary pattern that uniquely identifies the operation to be performed (e.g., ADD, LOAD, STORE, JUMP).
MAR (Memory Address Register)	A CPU register that holds the address of the memory location currently being accessed (read from or written to). Its contents are placed on the address bus during memory operations.
MDR (Memory Data Register)	A CPU register that holds the data being transferred to or from memory. Also called the Memory Buffer Register (MBR).
Stored-Program Concept	The foundational architectural principle (von Neumann architecture) that both program instructions and data are stored together in the same memory in binary form. This enables programmability — changing what the computer does requires only loading a different program.

Self Assessment — Questions

Section A: Multiple Choice Questions (MCQ)

1. Which type of computer is MOST suited for tasks such as weather forecasting and nuclear simulations that require hundreds of quadrillions of floating-point operations per second?

a) Mainframe Computer

b) Personal Computer

c) Supercomputer

d) Embedded System

2. The PARAM Siddhi-AI is an example of which type of computer, and which organisation developed it?

a) Mainframe; developed by IBM India

b) Supercomputer; developed by C-DAC, Pune

c) Minicomputer; developed by TCS

d) Embedded system; developed by ISRO

3. Which functional unit of the CPU is responsible for directing all other units by fetching and decoding instructions?

a) Arithmetic Logic Unit (ALU)

b) Memory Unit

c) Input Unit

d) Control Unit (CU)

4. The Program Counter (PC) register holds:

a) The current instruction being executed.

b) The address of the NEXT instruction to be fetched.

c) The data currently being processed by the ALU.

d) The result of the most recent arithmetic operation.

5. In the Fetch-Decode-Execute cycle, what happens IMMEDIATELY after an instruction is fetched from memory?

a) The Program Counter is reset to zero.

b) The instruction is written back to memory.

c) The instruction is decoded to determine the operation and operands.

d) The ALU performs the computation.

6. A 32-bit address bus can directly address a maximum of how much memory?

a) 32 MB

b) 64 MB

c) 2 GB

d) 4 GB

7. Which bus is BIDIRECTIONAL, allowing data to flow in BOTH directions between the CPU and memory?

a) Address Bus

b) Control Bus

c) Data Bus

d) All buses are bidirectional.

8. In a three-address instruction format 'ADD R3, R1, R2', what operation is performed?

a) R1 ← R1 + R2, R3 is unchanged.

b) R3 ← R1 + R3.

c) R3 ← R1 + R2, R1 and R2 are preserved.

d) R1 ← R2 + R3.

9. What is the decimal value of the binary number 10110110?

a) 172

b) 182

c) 162

d) 192

10. Which of the following is the correct hexadecimal equivalent of the binary number 11010101?

a) B5

b) D5

c) C4

d) E5

11. In 8-bit signed magnitude representation, what is the binary representation of -37?

a) 00100101

b) 11011010

c) 10100101

d) 11100101

12. What is the 8-bit 2's complement representation of -37?

a) 10100101

b) 11011011

c) 11011010

d) 10011010

13. A key advantage of 2's complement over signed magnitude is:

a) It uses more bits, allowing larger numbers.

b) There is only one representation of zero, and arithmetic requires no special sign handling.

c) It is easier to read by humans.

d) It can represent fractional numbers.

14. Perform 8-bit 2's complement arithmetic: 00110010 + 11001101. What is the result and is there overflow?

a) 11111111, no overflow

b) 00000000 with carry = 1, no overflow

c) 11111111, overflow

d) 100000000 (9 bits), which represents 256 decimal

15. Overflow occurs in 2's complement addition when:

a) The carry out of the MSB is 0.

b) Both operands are of different signs.

c) The carry INTO the MSB differs from the carry OUT of the MSB.

d) The result has more than 8 bits.

16. In LITTLE-ENDIAN byte ordering, when the 32-bit value 0xABCD1234 is stored starting at address 2000, which byte is at address 2000?

a) 0xAB (Most Significant Byte)

b) 0xCD

c) 0x12

d) 0x34 (Least Significant Byte)

17. The Memory Address Register (MAR) is used to:

a) Store the data read from memory.

b) Store the instruction currently being executed.

c) Hold the address of the memory location to be accessed.

d) Count the number of memory accesses performed.

18. During a memory WRITE operation, which of the following is the correct sequence of events?

a) CPU places address in MAR, places data in MDR, asserts Memory Read signal.

b) CPU places address in MAR, places data in MDR, asserts Memory Write signal — memory stores the data.

c) CPU places data in MDR, reads address from memory, asserts Memory Write.

d) Memory reads the address from MDR and the data from MAR.

19. Which processor architecture — CISC or RISC — typically uses three-address instructions where the result is stored in a separate destination register, preserving both source operands?

a) CISC (Complex Instruction Set Computer)

b) RISC (Reduced Instruction Set Computer)

c) Both CISC and RISC use identical instruction formats.

d) Neither; three-address instructions are used only in mainframes.

20. The stored-program concept, which forms the basis of all modern computers, was proposed in a 1945 report by which mathematician?

a) Alan Turing

b) Charles Babbage

c) John von Neumann

d) Ada Lovelace

Section B: True / False

State whether the following statements are True or False. (5 Questions)

1. Embedded computers are general-purpose computers that can be reprogrammed simply by loading a different program, just like a personal computer.

2. In the Fetch-Decode-Execute cycle, the Program Counter (PC) is incremented DURING the Fetch phase so that it points to the next instruction.

3. The address bus in a computer is bidirectional — addresses can be sent both from the CPU to memory and from memory to the CPU.

4. To convert a binary number to hexadecimal, group the binary digits into sets of 4 bits from the RIGHT (least significant end) and replace each group with its hexadecimal digit.

5. In 8-bit 2's complement representation, the value 10000000 represents -128 (not -0).

Section C: Short Answer Questions

Answer each question in 4–6 sentences or with clearly laid-out working steps. (5 Questions)

1. Explain the roles of the ALU and the Control Unit in the CPU. How do they work together to execute an instruction? Use the ADD R1, R2 instruction as an example to trace the execution.

2. Perform the following number system conversions, showing all working steps: (a) Convert 137 (decimal) to binary. (b) Convert 10101111 (binary) to hexadecimal. (c) Convert 3A (hexadecimal) to decimal.

3. Represent +75 and -75 in 8-bit signed magnitude, 8-bit 1's complement, and 8-bit 2's complement. Show all working steps clearly.

4. Using 8-bit 2's complement arithmetic, compute 83 - 47. Show: (a) the binary representation of both numbers, (b) the 2's complement of 47, (c) the addition, and (d) the decimal verification. Also determine whether overflow occurs.

5. Explain the difference between big-endian and little-endian byte ordering. Show how the 32-bit hexadecimal value 0x1A2B3C4D would be stored at memory addresses 5000–5003 in both big-endian and little-endian formats.

Self Assessment — Answer Key

Section A: MCQ

Q.No	Correct Answer / Explanation
1	c) Supercomputer — designed for maximum computation speed, measured in petaflops. Used for scientific simulations, weather forecasting, and AI research.
2	b) Supercomputer; developed by C-DAC (Centre for Development of Advanced Computing), Pune. PARAM Siddhi-AI is ranked among the global top 500 supercomputers.
3	d) Control Unit (CU) — it fetches instructions from memory, decodes the opcode, and issues control signals to all other units. The ALU performs computations but does not direct other units.
4	b) The address of the NEXT instruction to be fetched. The PC is incremented after each fetch so it always points ahead to the next instruction in sequence.
5	c) The instruction is decoded to determine the operation and operands. After the fetch, the Control Unit reads the instruction in the IR and decodes the opcode and operand addresses before proceeding to execute.
6	d) 4 GB. Maximum addressable memory = 2^32 = 4,294,967,296 bytes = 4 GB.
7	c) Data Bus — data must flow in both directions: from memory to CPU (read) and from CPU to memory (write). The address bus is unidirectional (CPU → memory only).

8	c) R3 ← R1 + R2. In a three-address format, the destination is separate from the sources, and both source registers R1 and R2 are preserved.
9	b) 182. 10110110 = 128+32+16+4+2 = 182. Check: 1×128 + 0×64 + 1×32 + 1×16 + 0×8 + 1×4 + 1×2 + 0×1 = 182.
10	b) D5. Group: 1101 | 0101. 1101 = 13 = D; 0101 = 5. Therefore 11010101 = D5 (hex).
11	c) 10100101. In signed magnitude: MSB=1 (negative), magnitude of 37 = 0100101. So -37 = 10100101.
12	b) 11011011. +37 = 00100101. 1's complement = 11011010. 2's complement = 11011010 + 1 = 11011011.
13	b) Only one representation of zero; arithmetic requires no special sign handling. 2's complement's greatest practical advantage is that standard binary addition hardware correctly handles both positive and negative number arithmetic.
14	a) 11111111, no overflow. 00110010 (50) + 11001101 (-51 in 2's complement) = 11111111 = -1 (decimal). Signs differ (positive + negative), so overflow is impossible. Carry-in = carry-out = 0 for MSB. No overflow flag set.
15	c) The carry INTO the MSB differs from the carry OUT of the MSB. This is the hardware test for overflow: if these two carries are different, the result has overflowed the signed representation range.
16	d) 0x34 (Least Significant Byte). In little-endian, the LSB is at the lowest address. 0xABCD1234: bytes are AB, CD, 12, 34. LSB = 0x34 is at address 2000.

17	c) Hold the address of the memory location to be accessed. The MAR contents are placed on the address bus to select the memory location for read or write.
18	b) CPU places address in MAR, places data in MDR, asserts Memory Write signal — memory stores the data. The MW control signal tells memory to accept the data on the data bus and store it at the address on the address bus.
19	b) RISC — Reduced Instruction Set Computer architectures (ARM, MIPS) typically use three-address instructions to preserve source operands. CISC (Intel x86) more commonly uses two-address instructions where one source is also the destination.
20	c) John von Neumann — his 1945 draft report on the EDVAC computer described the stored-program architecture. This is why the standard computer architecture model is called "von Neumann architecture".

Section B: True / False Answer Key

Q.No	Answer / Explanation
1	False — Embedded computers are SPECIAL-PURPOSE computers running a fixed program stored in non-volatile memory. Unlike general-purpose computers, they cannot be easily reprogrammed by simply loading a new program. They are designed for one specific function (e.g., controlling a washing machine cycle).
2	True — During the Fetch phase: the PC value is copied to the MAR, the instruction is fetched from memory, and then the PC is incremented. This

	ensures the PC always points to the NEXT instruction before the current one is decoded and executed.
3	False — The address bus is UNIDIRECTIONAL — addresses flow only FROM the CPU TO memory (or I/O devices). The CPU generates memory addresses; memory does not send addresses back to the CPU.
4	True — Grouping from the RIGHT (LSB side) ensures correct alignment. The rightmost group may need to be padded with leading zeros if it has fewer than 4 bits. This works because 16 = 2^4, so each hex digit exactly represents 4 binary digits.
5	True — In 8-bit 2's complement, 10000000 is a special case: it represents the most negative number, -128. This is because: invert 10000000 → 01111111 = 127; add 1 → 10000000 = 128. So -128 wraps back to 10000000, and there is no +128 in 8-bit 2's complement. Range is -128 to +127.

Section C: Short Answer — Suggested Answers

Q.No	Suggested Answer
1	The Control Unit (CU) manages the CPU's operation — it fetches instructions from memory, decodes the opcode, and issues control signals. It does NOT perform calculations. The ALU performs all arithmetic and logical computations. For ADD R1, R2: (Fetch) PC points to the instruction; it is fetched into the IR, PC incremented. (Decode) CU reads IR: opcode=ADD, source=R2, destination=R1; CU signals ALU for addition and routes R1, R2 to ALU inputs. (Execute) ALU computes R1+R2; result stored

	in R1; status flags updated. The CU directs, the ALU calculates — neither unit functions alone.
2	(a) 137 to binary: 137÷2=68 r1; 68÷2=34 r0; 34÷2=17 r0; 17÷2=8 r1; 8÷2=4 r0; 4÷2=2 r0; 2÷2=1 r0; 1÷2=0 r1. Read bottom to top: 10001001. Check: 128+8+1=137 ✓. (b) 10101111 to hex: group → 1010|1111 = A|F = 0xAF. (c) 3A hex to decimal: 3×16+10 = 48+10 = 58 decimal.
3	+75 = 01001011 (positive, MSB=0, same in all three methods). -75: Signed Magnitude: 11001011 (MSB=1, magnitude=75). 1's Complement: invert +75 → 10110100. 2's Complement: 10110100 + 1 = 10110101. Verification: 10110101 (2's comp) = -(01001011) = -(64+8+2+1) = -75 ✓.
4	83 = 01010011. 47 = 00101111. 2's complement of 47: invert → 11010000, +1 → 11010001. Add: 01010011 + 11010001 = 100100100. Discard carry (9th bit): result = 00100100 = 32+4 = 36. Verification: 83-47=36 ✓. Overflow check: carry-in to MSB and carry-out from MSB are both 1 (same) → NO OVERFLOW. Result is correct.
5	Big-endian stores the Most Significant Byte at the lowest address; little-endian stores the Least Significant Byte first. For 0x1A2B3C4D (bytes: 1A, 2B, 3C, 4D): Big-endian → Address 5000:1A, 5001:2B, 5002:3C, 5003:4D. Little-endian → Address 5000:4D, 5001:3C, 5002:2B, 5003:1A. Intel x86/x64 processors (most PCs) are little-endian. Network protocols (TCP/IP) use big-endian (network byte order). When communicating between systems with different byte orders, bytes must be explicitly swapped using functions like htons() and ntohl() in C networking code.

✓ Let's Check 3	
Q.No	**Question**
1	MCQ: What is the 8-bit 2's complement of the decimal value -100? a) 10100000 b) 11001100 c) 10011100 d) 11100100
2	MCQ: Which of the following is the CORRECT binary result of adding 01100111 (103) + 00111000 (56) in 8-bit arithmetic, and does overflow occur? a) 10011111 (= -97 in 2s complement); OVERFLOW occurs. b) 10011111 (= 159 unsigned); no overflow. c) 01001111 (= 79); no overflow. d) 11100001; overflow occurs.
3	Fill in the Blank: The control signal that tells the memory module to place data onto the data bus so the CPU can read it is called the Memory _____ signal.
4	Fill in the Blank: In big-endian byte ordering, the _____ significant byte is stored at the lowest memory address.
5	True/False: In 8-bit 2's complement arithmetic, the maximum positive value representable is 127 (01111111), and the minimum negative value is -128 (10000000).
6	True/False: A supercomputer is simply a faster version of a personal computer and uses the same type of processor chips found in consumer laptops.

Answers	
1	c) 10011100. +100 = 01100100. 1's complement = 10011011. 2's complement = 10011011 + 1 = 10011100. Verify: 10011100 → invert = 01100011 = 99, + 1 = 100. So 10011100 = -100 ✓.
2	a) 10011111; OVERFLOW occurs. 01100111 + 00111000: both are positive (MSB=0). Sum = 10011111. MSB of result = 1 → sign changed from positive to negative. Carry-in to MSB (bit 7) = 1, carry-out = 0. They differ → OVERFLOW. Result 10011111 represents -97 in 2s complement, but 103+56=159 which exceeds 8-bit max of +127.
3	Read (Memory Read / MR). The MR control signal causes the memory to read the data at the specified address and place it on the data bus for the CPU to receive into the MDR.
4	Most. In big-endian, the most significant byte (the byte with the highest positional weight) is placed at the lowest address, mirroring human reading order (left to right = most to least significant).
5	True — 8-bit 2's complement range is -2^7 to +(2^7 - 1) = -128 to +127. 01111111 = 127 (max positive); 10000000 = -128 (most negative, a special case with no positive counterpart in 8-bit).
6	False — Supercomputers are architecturally very different from personal computers. They use thousands to hundreds of thousands of specialised processors (often custom-designed CPUs or GPUs) working in parallel through high-speed interconnects, managed by specialised operating systems. A consumer laptop has 4–16 processor cores; a top supercomputer may have over a million.

JGI JAIN DEEMED-TO-BE UNIVERSITY | ONLINE

ONLINE DEGREE PROGRAMS OFFERED

B.Com | BBA | MBA | MCA | M.Com | MA